

AI Cloud Ops Platform

Technical Project Report

Full-Stack AI-Powered Azure Cloud Operations Dashboard

Version 1.0

June 2025

Table of Contents

Table of Contents.....	2
1. Abstract.....	4
2. Introduction	4
3. Project Objectives	5
4. Technology Stack	6
4.1 Frontend.....	6
4.2 Backend	6
4.3 AI Technologies.....	6
4.4 Azure Services	7
5. System Architecture	8
5.1 Frontend Architecture	8
5.2 Backend Architecture.....	8
5.3 AI Orchestration Architecture.....	9
5.4 Azure Integration Architecture	9
6. Data Flow.....	10
6.1 Resource Inventory Flow	10
6.2 AI Query Processing Flow	11
6.3 Action Approval Flow	11
6.4 Session Credential Flow	12
7. AI Agent Architecture	13
7.1 Router Agent	13
7.2 Tool Selector	13
7.3 Operational Agent.....	14
7.4 Security Agent	14
7.5 FinOps Agent	14
7.6 General Agent	14
8. Azure Integration.....	16
8.1 Azure Resource Graph	16
8.2 Log Analytics	16
8.3 Azure Advisor	16
8.4 Cost Management	16
8.5 Container Apps.....	16
9. Security & Governance	17
9.1 Session-Based Credentials	17
9.2 Approval Gates.....	17

9.3 Resource Group Restrictions.....	17
9.4 Human-in-the-Loop Model	17
10. Key Features Implemented	18
11. Challenges & Solutions	19
12. Future Enhancements	20
13. Conclusion	21

1. Abstract

The AI Cloud Ops Platform is a full-stack, AI-powered cloud operations dashboard purpose-built for Microsoft Azure environments. The system integrates a multi-agent AI architecture with real-time Azure telemetry to enable natural language querying, intelligent resource analysis, and governed write-back operations. By combining a Next.js frontend with a FastAPI backend, LangGraph-orchestrated agents, and direct Azure SDK integration, the platform provides cloud engineers and FinOps practitioners with a unified interface for operational visibility, security posture review, and cost optimization.

2. Introduction

Modern cloud environments generate vast telemetry across compute, networking, cost, and security domains. Traditional dashboards expose raw metrics but rarely surface actionable intelligence. The AI Cloud Ops Platform addresses this gap by embedding a multi-agent AI layer directly between the user and Azure's native APIs.

The platform enables engineers to interrogate their infrastructure using natural language, receive specialist AI analysis across operational, security, and FinOps domains, and execute approved write-back actions — all within a single browser-based interface. Human-in-the-loop approval gates ensure no modification reaches Azure without explicit operator consent.

3. Project Objectives

- Deliver real-time Azure resource inventory through Resource Graph API with live polling and tag-based filtering.
- Enable natural language cloud operations queries routed through a specialized multi-agent AI system.
- Implement structured approval gates for all write-back operations to Azure, maintaining a full audit trail.
- Integrate Azure Log Analytics, Advisor, and Cost Management APIs to surface health, security, and cost intelligence.
- Provide session-based credential management supporting both environment-variable and runtime-supplied Azure service principal credentials.
- Deploy the entire stack via Docker Compose with containerized frontend and backend workloads hosted on Azure Container Apps.

4. Technology Stack

The platform is built on a deliberately lean, modern stack. Each technology was selected for production readiness, community support, and suitability to AI-augmented cloud operations workloads.

4.1 Frontend

Technology	Version	Purpose & Justification
Next.js	16.2.6	React-based framework providing App Router, server components, and optimized client-side rendering for a responsive cloud dashboard.
React	19.2.4	Component model enables fine-grained reactivity for live-polling hooks (useCloudResources, useChat) without full-page refreshes.
TypeScript	5.x	Static typing across frontend services, hooks, and component props reduces runtime errors in a data-intensive dashboard.
Tailwind CSS	v4	Utility-first styling enables rapid, consistent UI development without a separate design system or CSS-in-JS overhead.
Recharts	2.x	Declarative charting library renders CloudIntelligence analytics panels with minimal configuration and React integration.

4.2 Backend

Technology	Version	Purpose & Justification
FastAPI	0.115.x	Async Python web framework on port 8000. Chosen for OpenAPI auto-documentation, Pydantic schema validation, and high-throughput async handlers suited to Azure SDK I/O.
MCP Server (FastAPI)	Custom	Lightweight model context protocol server on port 8001. Acts as a sandboxed tool execution layer, isolating Azure SDK calls from the main agent graph.
Uvicorn	0.30.x	ASGI server for both FastAPI instances, enabling async request handling and hot-reload in development.
Pydantic / Schemas	2.x	Typed schema definitions in models/schemas.py enforce request and response contracts across all REST endpoints.
SQLite (actions.db)	Built-in	Lightweight persistent store for action history. Current implementation uses in-memory action_store.py; SQLite file retained for future migration.

4.3 AI Technologies

Technology	Detail	Purpose & Justification
Groq (llama-3.1-8b-instant)	Inference API	Ultra-low-latency inference for three sequential Groq calls per user query (router, tool selector, specialist). The 8B Llama model balances reasoning quality with sub-second response times critical for interactive chat.
LangGraph	0.2.x	Stateful agent graph orchestration. Models the multi-agent workflow as a directed graph, enabling deterministic routing from router node through tool selection to specialist agent.
Model Context Protocol (MCP)	Custom impl.	Tool invocation protocol separating agent reasoning (graph.py) from Azure SDK execution (mcp_server/). Enables clean tool registration via registry.py and isolated testability.

4.4 Azure Services

Service	SDK Module	Role
Azure Resource Graph	azure.mgmt.resourcegraph	Primary inventory source. KQL-based queries return resource-level metadata across subscription.
Log Analytics Workspace	azure.monitor.query	Health log ingestion and KQL query execution for error and operation failure analysis.
Azure Advisor	azure.mgmt.advisor	Security, reliability, and cost recommendation retrieval.
Cost Management	azure.mgmt.costmanagement	Monthly, daily, and resource-group-level spend retrieval with budget status and anomaly detection.
Container Apps API	azure.mgmt.appcontainers	Stop and start Container App workloads as write-back actions post-approval.
Azure Resource Manager	azure.mgmt.resource / compute / web	Tag application, VM lifecycle management, and App Service plan scaling.

5. System Architecture

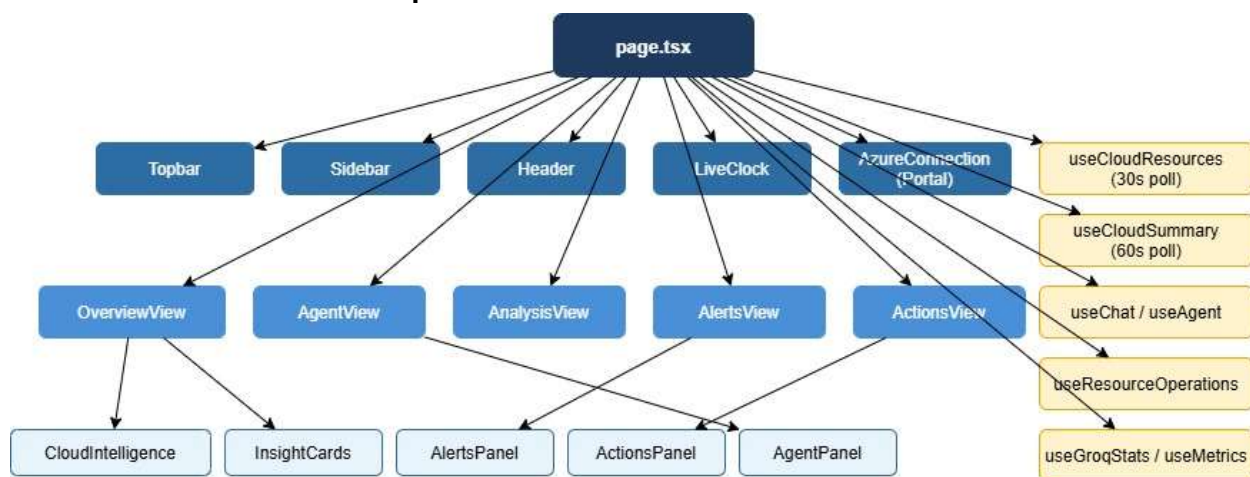
The platform is structured across four architectural layers: a React frontend, a FastAPI backend, an MCP tool execution layer, and the Azure cloud plane. These layers interact through clearly defined REST and internal HTTP interfaces.

5.1 Frontend Architecture

The frontend is a Next.js 16 application structured around a single root page (`app/page.tsx`) that composes the full dashboard. The view layer is split into five named views — Overview, Agent, Analysis, Alerts, and Actions — each rendered conditionally based on navigation state. Data is exclusively sourced through custom React hooks, which abstract all REST polling and chat streaming behind stable interfaces.

Key hooks include `useCloudResources` (polls `/cloud-resources` every 30 seconds), `useCloudSummary` (polls `/cloud-summary` every 60 seconds), and `useResourceOperations` (dispatches write-back requests). The `AgentPanel` component manages chat state through `useChat` and `useAgent`, maintaining message history and streaming AI responses. The `ActionsPanel` polls `/actions` every 5 seconds to surface the agent-proposed action queue for operator approval.

Title: Frontend Architecture — Component & Data Flow



5.2 Backend Architecture

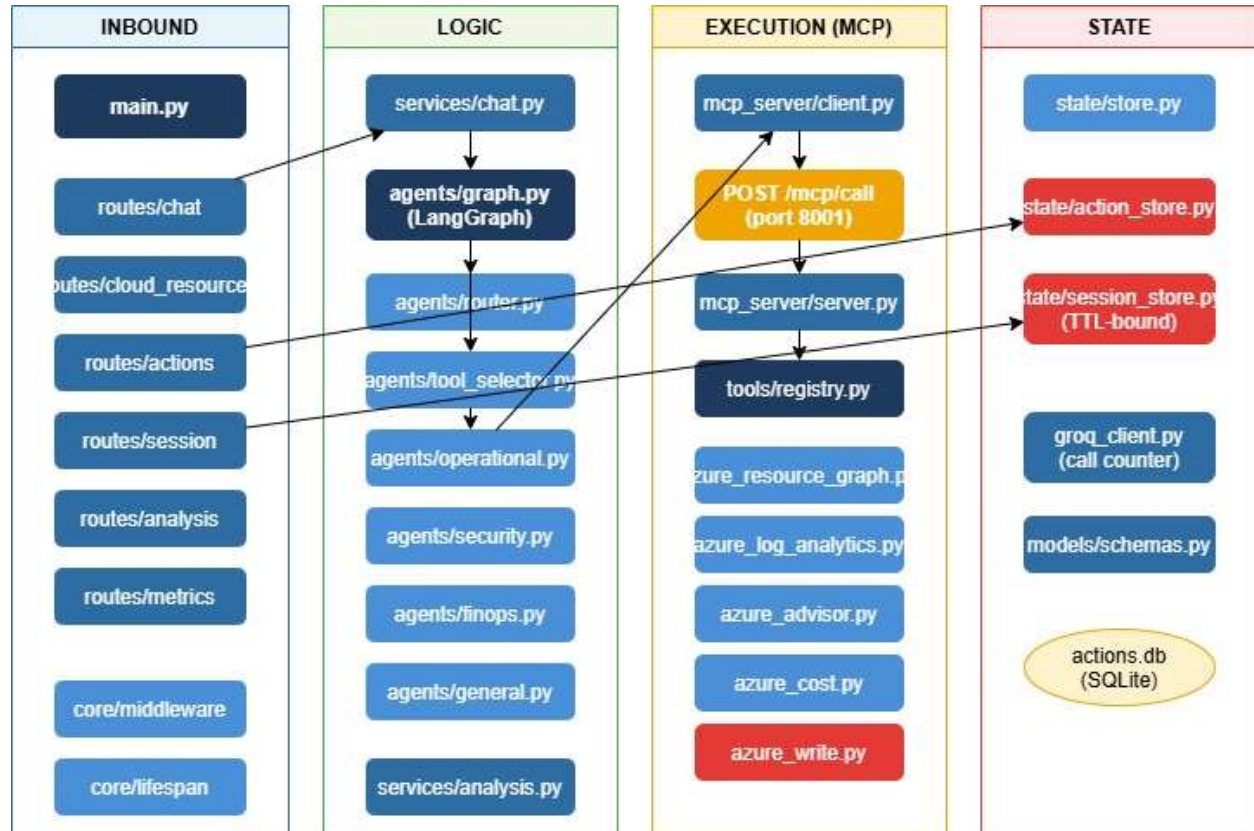
The FastAPI backend (port 8000) is organized into `routes/`, `services/`, `agents/`, `tools/`, `state/`, and `mcp_server/` packages. The lifespan module (`core/lifespan.py`) handles startup initialization. All inbound requests pass through middleware in `core/middleware.py` before reaching route handlers.

Route handlers in `routes/` are thin: they parse requests, delegate to `services/` or `agents/` for business logic, and return typed Pydantic responses. The `state/` package maintains three in-memory stores: `store.py` (general), `action_store.py` (pending/approved/rejected actions), and

session_store.py (TTL-bound Azure credentials). The MCP server runs as a second FastAPI process on port 8001, exposing a /mcp/call endpoint consumed exclusively by mcp_server/client.py from within the main process.

Diagram Description — Backend Package Architecture

Title: Backend Service Architecture



5.3 AI Orchestration Architecture

The agent orchestration layer is implemented as a LangGraph directed graph in `agents/graph.py`. Each user chat message triggers a deterministic three-node execution sequence: the Router node classifies intent, the Tool Selector node selects 1 to 4 appropriate tools, and the Specialist node executes the query with tool results. General queries bypass tool selection entirely and receive a direct Groq response.

5.4 Azure Integration Architecture

All Azure API interactions are encapsulated within the `tools/` package. Each tool module (`azure_resource_graph.py`, `azure_log_analytics.py`, `azure_advisor.py`, `azure_cost.py`, `azure_write.py`) resolves credentials at call time by checking `session_store.py` first, then falling back to environment variables. This dual-credential model allows runtime subscription switching without process restarts. The `azure_write.py` module enforces resource group allowlisting (`AZURE_ALLOWED_RGS`) before executing any mutation.

6. Data Flow

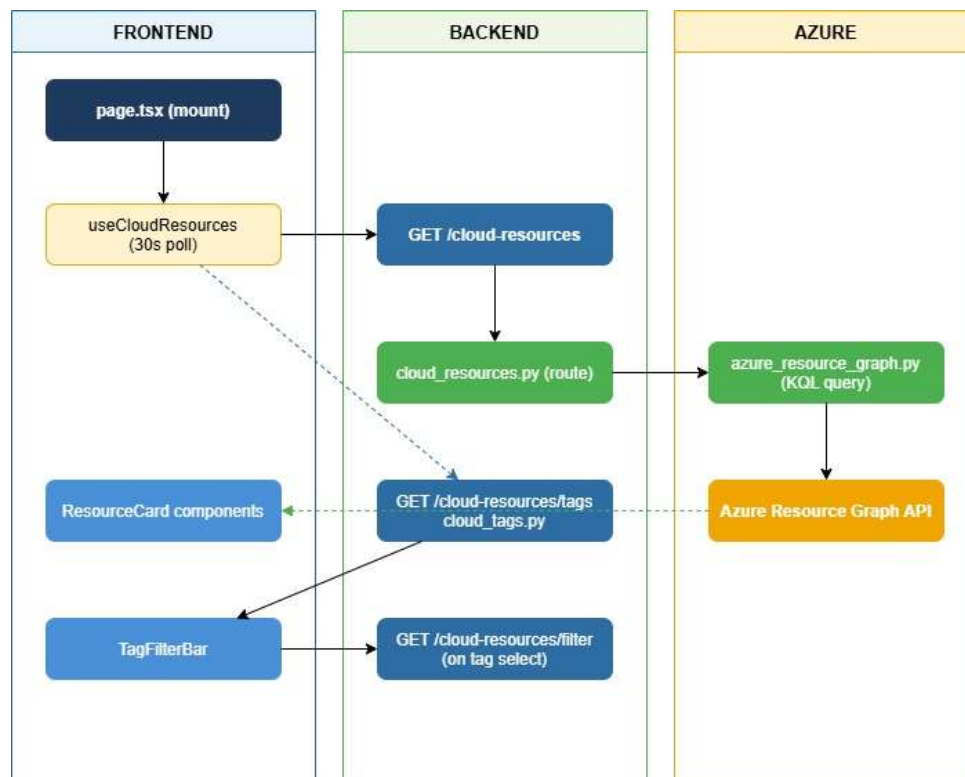
Four primary data flows govern platform operation. Each flow is described with its sequence of operations and a diagram description suitable for reproduction in Draw.io.

6.1 Resource Inventory Flow

When the dashboard initializes or the operator triggers a manual refresh, `page.tsx` activates `useCloudResources`, which dispatches a GET request to `/cloud-resources` on the FastAPI backend. The route handler `cloud_resources.py` constructs an Azure Resource Graph KQL query scoped to the configured subscription and allowed resource groups. The `azure_resource_graph.py` tool module resolves credentials, instantiates a `ResourceGraphClient`, and executes the query, returning resource-level records including name, resource ID, resource group, type, location, tags, and provisioning state.

The response is parsed and cached in React state, populating `ResourceCard` components in `OverviewView`. Tag values extracted from all resources are simultaneously fetched via `/cloud-resources/tags` (`cloud_tags.py`) to populate the `TagFilterBar`. Subsequent filter interactions dispatch GET `/cloud-resources/filter` with selected tag parameters without re-fetching the full inventory. The hook polls automatically every 30 seconds, preserving active filter state across cycles.

Diagram Description

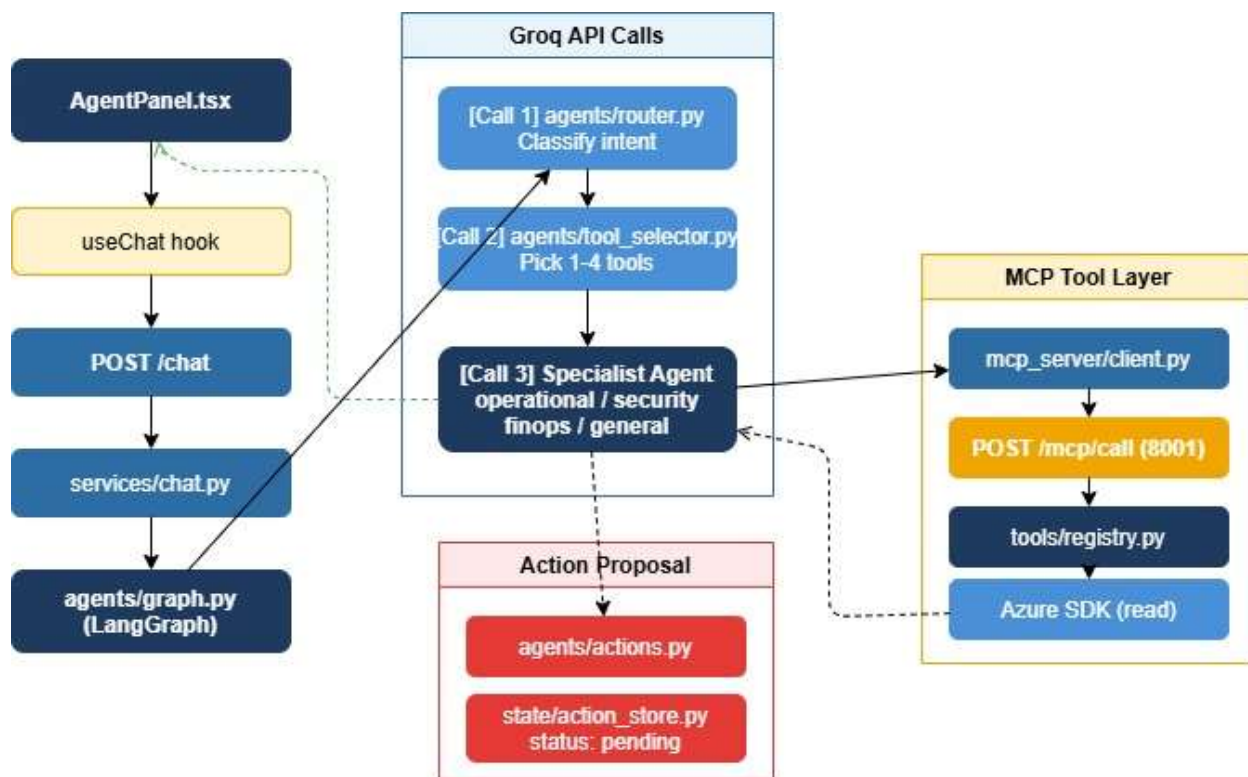


6.2 AI Query Processing Flow

The user submits a natural language query through AgentPanel.tsx. The component calls useChat, which POSTs the message to /chat on the FastAPI backend. The chat route handler delegates to services/chat.py, which invokes the LangGraph agent graph defined in agents/graph.py.

The graph executes three sequential Groq API calls. In the first call, agents/router.py classifies the query intent into one of four categories: operational, security, finops, or general. In the second call, agents/tool_selector.py receives the classified intent and selects between one and four relevant tools from the agent's tool registry. In the third call, the specialist agent (operational.py, security.py, finops.py, or general.py) is invoked with the selected tools and formulates a response.

When tool execution is required, the specialist calls mcp_server/client.py, which POSTs a tool invocation request to the MCP server at port 8001. The MCP server resolves the tool name against tools/registry.py and executes the corresponding Azure SDK function, returning structured results to the specialist. The specialist synthesizes tool outputs with its Groq response and returns the final answer through the chain to AgentPanel for display. If the specialist identifies an actionable finding, it additionally calls agents/actions.py to enqueue a proposed action.



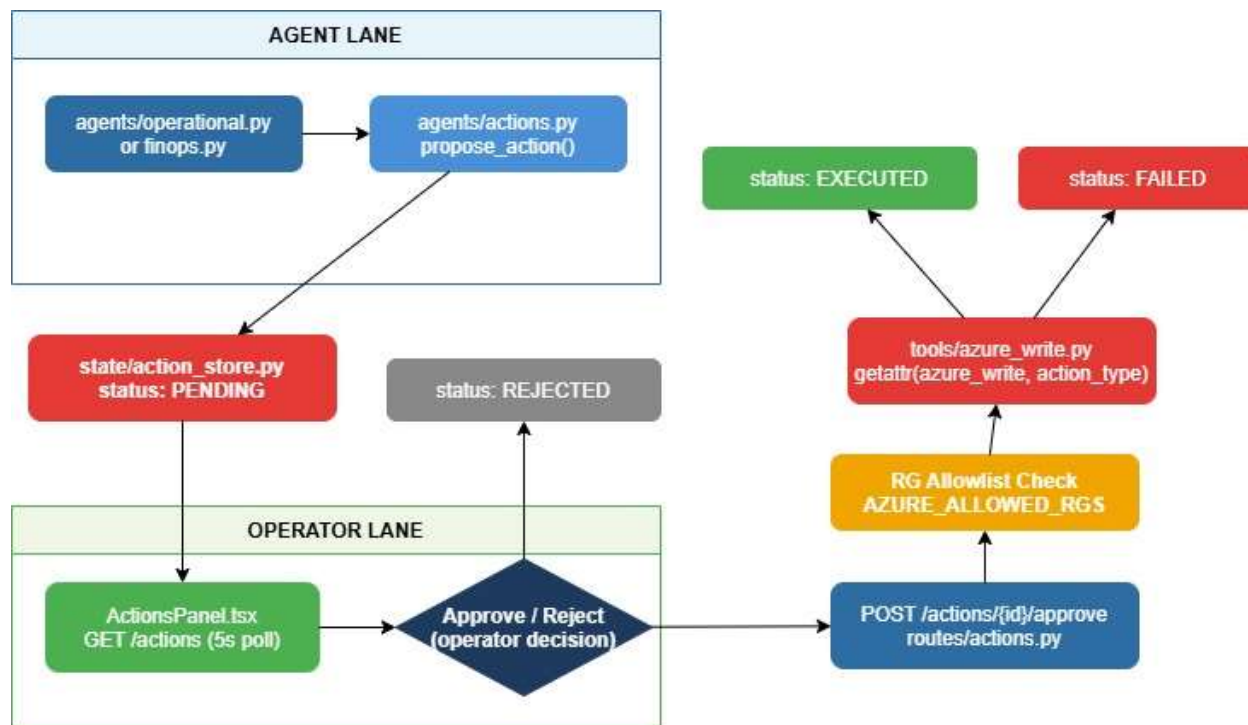
6.3 Action Approval Flow

When an agent proposes an action (such as applying tags to an untagged resource or stopping an idle Container App), agents/actions.py calls propose_action(), which enqueues the action in

state/action_store.py with a status of pending. ActionsPanel.tsx polls GET /actions every 5 seconds and displays all pending actions in the approval queue.

The operator reviews the proposed action — which includes the action type, target resource, and parameters — and clicks Approve or Reject. An Approve interaction sends POST /actions/{id}/approve to the backend. The actions route handler retrieves the action record, validates the target resource group against AZURE_ALLOWED_RGS, and uses dynamic dispatch (getattr(azure_write, action_type)) to call the appropriate write function in tools/azure_write.py. The write function invokes the Azure management SDK with the operator-supplied service principal credentials. On success, the action status transitions to executed. On failure, it transitions to failed. A Reject interaction sends POST /actions/{id}/reject, which marks the action rejected without touching Azure.

Diagram Description



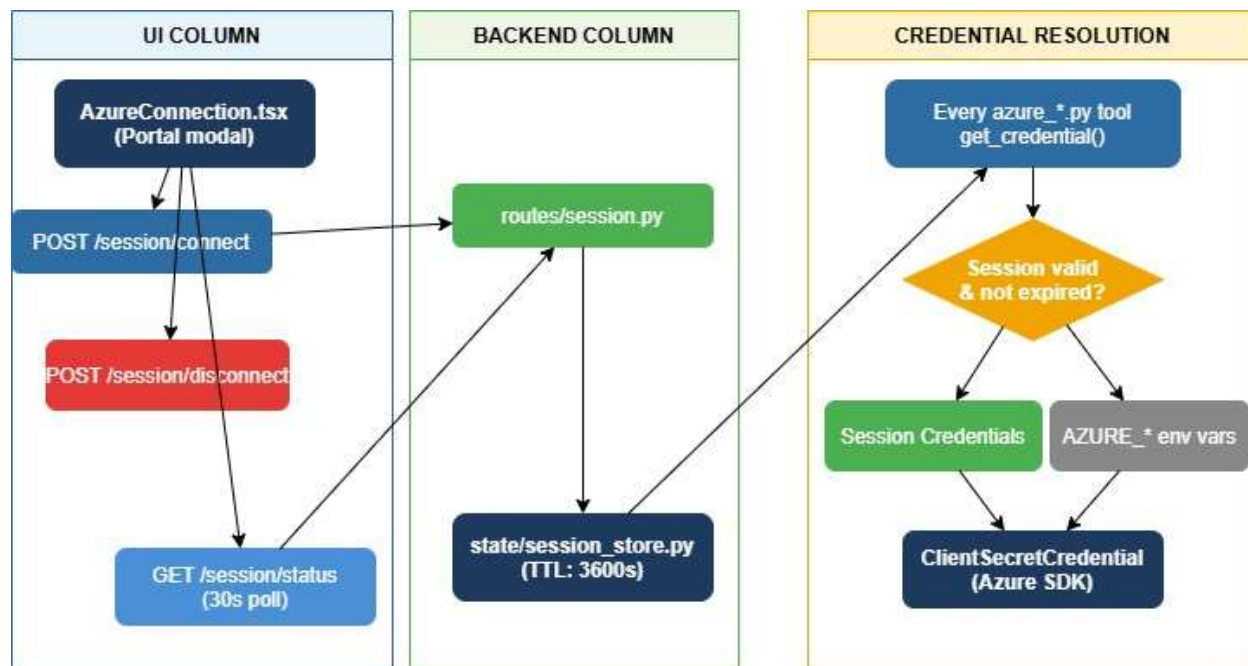
6.4 Session Credential Flow

The AzureConnection modal (rendered via React createPortal to avoid z-index clipping) allows operators to supply Azure service principal credentials at runtime: subscription ID, tenant ID, client ID, and client secret. On submission, the modal POSTs to /session/connect. The session route handler stores the credentials in session_store.py as an in-memory record with a TTL of 3600 seconds (configurable via AZURE_SESSION_TTL_SECONDS).

Every Azure tool module (azure_resource_graph.py, azure_log_analytics.py, etc.) calls session_store.get_credential() at the start of each invocation. If a valid, non-expired session record is present, the session credentials are used to construct the Azure ClientSecretCredential. If no session is active or the TTL has elapsed, the tool falls back to the

AZURE_* environment variables. The AzureConnection component polls /session/status every 30 seconds to reflect the live TTL countdown in the UI. Operators can explicitly terminate sessions via POST /session/disconnect.

Diagram Description



7. AI Agent Architecture

The agent system is implemented as a LangGraph graph (agents/graph.py) where each node represents an agent or selector. All agent-to-Groq communication is mediated through services/groq_client.py, which wraps the Groq SDK with a call counter exposed via the /metrics endpoint.

7.1 Router Agent

File: agents/router.py — Groq Call 1 of 3 per query.

The Router receives the raw user message and classifies it into one of four intents: operational, security, finops, or general. It outputs a structured classification that the LangGraph graph uses to determine the next node. The classification prompt includes intent definitions and example phrasings to minimize misrouting.

7.2 Tool Selector

File: agents/tool_selector.py — Groq Call 2 of 3 per query.

The Tool Selector receives the classified intent and the user's original query and outputs a list of 1 to 4 tool names from the specialist agent's available tool registry. Dynamic tool selection

avoids unnecessary Azure API calls and reduces latency by fetching only the data required to answer the specific query.

7.3 Operational Agent

File: agents/operational.py — Groq Call 3 for operational queries.

Responsible for infrastructure health and resource management. Available tools include `get_resource_inventory`, `get_unhealthy_resources`, `get_resource_group_summary`, `get_log_summary`, `get_recent_errors`, `get_failed_operations`, `get_resource_health_logs`, `get_full_resource_report`, and `get_resource_details`. Proposes write-back actions such as stopping idle Container Apps via `agents/actions.py`.

7.4 Security Agent

File: agents/security.py — Groq Call 3 for security queries.

Focuses on governance and configuration risk. Tools include `get_untagged_resources`, `get_recently_modified_resources`, `get_advisor_security_recommendations`, `get_advisor_reliability_recommendations`, `get_resource_health_logs`, `get_failed_operations`, and `get_full_resource_report`. Proposes `apply_tags` actions for resources lacking mandatory tags.

7.5 FinOps Agent

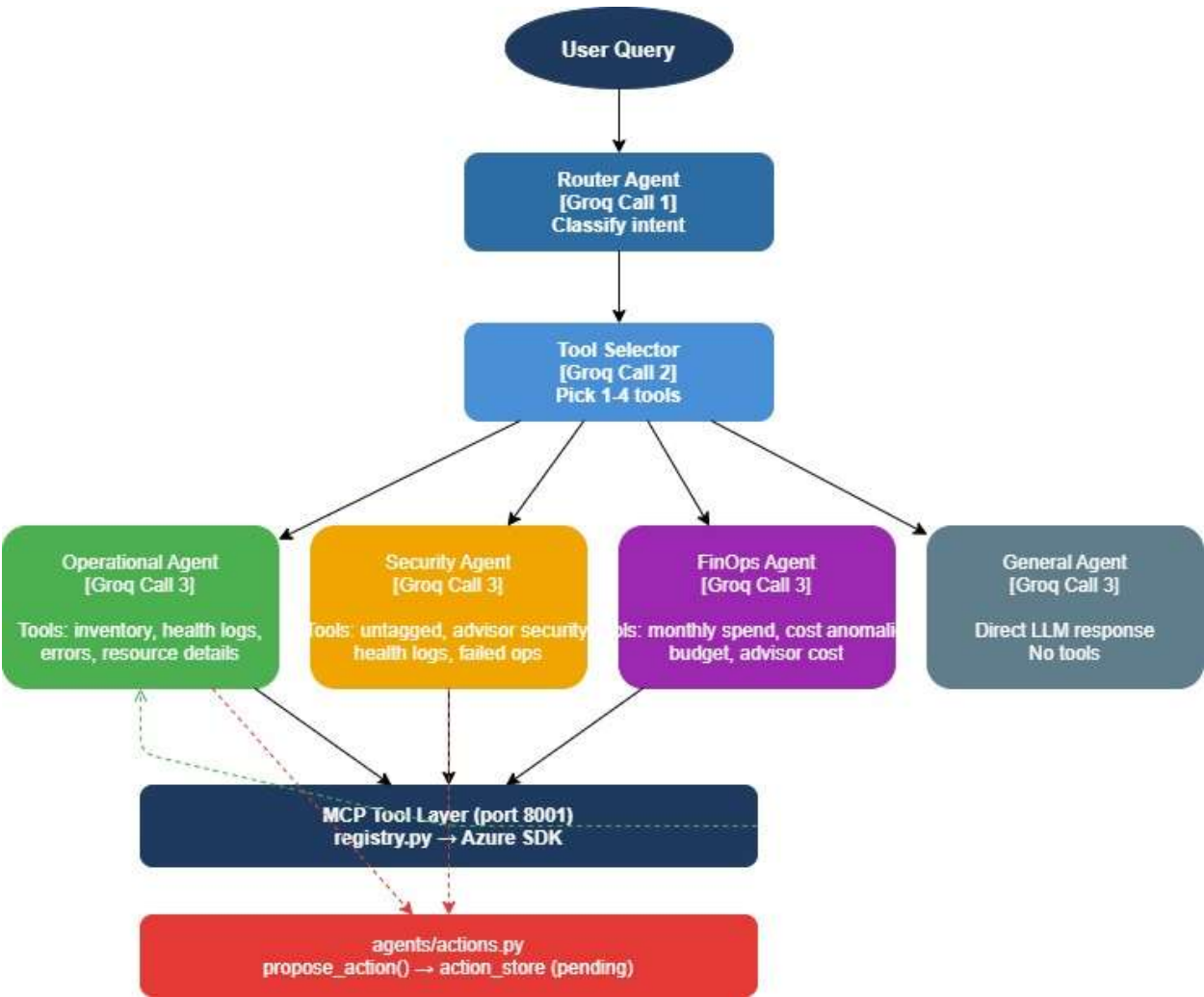
File: agents/finops.py — Groq Call 3 for cost queries.

Analyzes cloud spend and recommends cost optimizations. Tools include `get_monthly_spend`, `get_daily_spend`, `get_cost_by_resource_group`, `get_budget_status`, `get_cost_anomalies`, `get_full_cost_report`, `get_advisor_cost_recommendations`, `get_untagged_resources`, and `get_full_resource_report`. Cost hallucination guardrails were added to prevent the model from fabricating spend figures when Cost Management API returns restricted data on the student subscription.

7.6 General Agent

File: agents/general.py — Groq Call 3 for general queries.

Handles conversational queries that do not require Azure data retrieval. Invokes Groq directly without tool calls, returning natural language responses about cloud operations concepts, platform usage guidance, or contextual explanations.



Agent	Groq Call	Tool Access	Write-back
Router	Call 1	None — classification only	No
Tool Selector	Call 2	None — selection only	No
Operational	Call 3	9 read tools	Yes — stop/start Container App
Security	Call 3	8 read tools	Yes — apply_tags
FinOps	Call 3	9 read tools	No (propose only)
General	Call 3	None — direct LLM	No

8. Azure Integration

8.1 Azure Resource Graph

File: `tools/azure_resource_graph.py`. The Resource Graph API is the backbone of the inventory system. KQL queries return resource-level metadata across the entire subscription, scoped to allowed resource groups. The tool exposes `get_resource_inventory` (full list with state), `get_resource_details` (lookup by name), `get_resource_group_summary` (aggregated counts), `get_untagged_resources`, `get_recently_modified_resources`, and `get_unhealthy_resources`. Resource IDs and names are preserved in all responses to enable accurate downstream tool chaining.

8.2 Log Analytics

File: `tools/azure_log_analytics.py`. The Log Analytics integration queries the configured workspace via KQL against the `AzureActivity` and `AzureDiagnostics` tables. Functions include `get_log_summary` (activity overview), `get_recent_errors` (error-level events), `get_failed_operations` (failed ARM operations), and `get_resource_health_logs` (per-resource health events with `ResourceId` and `ResourceName` fields). The workspace ID is supplied via environment variable.

8.3 Azure Advisor

File: `tools/azure_advisor.py`. The Advisor integration retrieves recommendations across three categories: security (`get_advisor_security_recommendations`), reliability (`get_advisor_reliability_recommendations`), and cost (`get_advisor_cost_recommendations`). Recommendations include the affected resource, recommendation description, and impact level, enriching both the InsightCards panel and the Security and FinOps agent responses.

8.4 Cost Management

File: `tools/azure_cost.py`. Cost Management queries retrieve monthly and daily spend aggregated by resource group. The FinOps agent uses `get_monthly_spend`, `get_daily_spend`, `get_cost_by_resource_group`, `get_budget_status`, `get_cost_anomalies`, and `get_full_cost_report`. On the student Azure subscription, Cost Management API access is partially restricted; cost hallucination guardrails in `agents/finops.py` detect empty API responses and prevent the model from generating fabricated cost figures.

8.5 Container Apps

File: `tools/azure_write.py`. The Container Apps integration provides `stop_container_app` and `start_container_app` write actions using the `azure.mgmt.appcontainers` SDK. The start action accepts an optional `max_replicas` parameter (default: 1). Dynamic API version resolution ensures compatibility with the Azure resource provider version in use. All Container App mutations are gated behind the approval workflow and resource group allowlist validation.

9. Security & Governance

9.1 Session-Based Credentials

Azure service principal credentials are never persisted to disk. The `session_store.py` module holds credentials exclusively in process memory with a configurable TTL (default: 3600 seconds). On session expiry or explicit disconnect, credentials are cleared from memory. The fallback to environment variables ensures the system remains operational in CI/CD and containerized deployments where runtime credential entry is impractical.

9.2 Approval Gates

The platform enforces a strict human-in-the-loop model for all write operations. No Azure management API mutation is called until an operator explicitly approves an action in the `ActionsPanel`. The REST action lifecycle (pending -> approved -> executed / failed / rejected) provides a complete audit trail. The dynamic dispatch pattern in `routes/actions.py` (`getattr(azure_write, action_type)`) ensures only registered, typed action functions can be invoked.

9.3 Resource Group Restrictions

All write actions in `azure_write.py` validate the target resource group against the `AZURE_ALLOWED_RGS` environment variable before execution. Actions targeting resource groups outside the allowlist are rejected at the API layer, preventing accidental or unauthorized modification of production resources outside the governed scope.

9.4 Human-in-the-Loop Model

The platform's AI agents are deliberately scoped to proposal, not execution. Agents analyze, recommend, and propose — operators decide and approve. This model ensures AI-generated insights augment rather than replace operator judgment. The `ActionsPanel` surfaces the agent's reasoning alongside the proposed action, enabling informed approval decisions with full context.

10. Key Features Implemented

Feature	Implementation Detail	Status
Live Resource Inventory	Azure Resource Graph KQL, 30-second polling, resource-level fields	Operational
Multi-Agent AI System	LangGraph graph, 3 Groq calls per query, 4 specialist agents	Operational
Dynamic Tool Selection	Tool Selector agent picks 1-4 tools per query from specialist registry	Operational
Action Approval Queue	In-memory action_store, REST endpoints, ActionsPanel UI	Operational
Session-Based Credentials	TTL-bound in-memory store, dual credential resolution	Operational
Tag Filtering	Backend filter endpoint, frontend TagFilterBar, filter state persisted across polls	Operational
Log Analytics Integration	KQL queries for errors, failed ops, and per-resource health logs	Operational
Azure Advisor Integration	Security, reliability, and cost recommendations surfaced in InsightCards	Operational
Write-Back Actions	apply_tags, stop/start_container_app, stop/start_vm, scale_app_service, delete_resource	Operational
Cost Hallucination Guardrails	FinOps agent detects empty Cost Management responses, suppresses fabrication	Operational
Groq Call Counter	useGroqStats hook polls /metrics, displays live inference call count	Operational
CloudIntelligence Panel	Dynamic Recharts visualizations of resource and cost data	Operational

11. Challenges & Solutions

Challenge	Root Cause	Solution Implemented
Cost Management API restrictions	Student Azure subscription limits Cost Management access	Added response-empty detection in finops.py; agent returns 'data unavailable' rather than hallucinated figures
Unhealthy resource false positives	Resource Graph provisioning states misclassified as unhealthy	Refined KQL filter logic in azure_resource_graph.py to exclude transient provisioning states
AzureConnection modal clipping	Modal rendered inside dashboard layout; z-index and overflow clipping prevented full display	Refactored to React createPortal, rendering modal at document body level
Filter state lost on poll	useCloudResources hook reset filter parameters on each 30-second poll cycle	Moved filter state outside the polling closure; filters preserved as stable React state across poll cycles
Dynamic API version for write actions	Azure Container Apps resource provider requires specific API versions not available in SDK defaults	azure_write.py performs runtime API version resolution via Resource Graph before issuing management calls
Agent tool over-fetching	Early implementation called all available tools for every query, increasing latency and token cost	Introduced tool_selector.py (Groq Call 2) to dynamically select 1-4 tools per query based on classified intent

12. Future Enhancements

- **Persistent Action History:** Migrate `action_store.py` from in-memory to SQLite or Redis to preserve the action audit trail across backend restarts.
- **Bulk Approve / Reject:** Extend `ActionsPanel` UI to support multi-select action operations, reducing operator friction for batch remediation tasks.
- **Azure Policy Enforcement:** Integrate Azure Policy API to surface compliance violations alongside Advisor recommendations, enabling policy-driven tagging and configuration actions.
- **Teams and Email Notifications:** Connect approval gate events to Microsoft Teams webhooks or Azure Communication Services email for asynchronous operator notification.
- **Enhanced FinOps Intelligence:** When Cost Management access is fully available, integrate budget forecasting and anomaly trend analysis with chart visualization in `CloudIntelligence`.
- **Multi-Subscription Support:** Extend `session_store.py` to support multiple concurrent subscription sessions, enabling cross-subscription resource comparison.
- **Agent Memory and Context:** Incorporate `LangGraph` checkpointing to retain conversation context across sessions, enabling follow-up queries without re-stating resource context.
- **Lint and CI Pipeline:** Resolve pre-existing frontend lint issues and establish a GitHub Actions pipeline for automated lint, type-check, and backend unit test execution.

13. Conclusion

The AI Cloud Ops Platform demonstrates a production-grade integration of multi-agent AI with native Azure cloud APIs, delivered through a clean full-stack architecture. The three-Groq-call agent pipeline — router, tool selector, specialist — provides accurate, low-latency responses without over-fetching Azure data. The MCP server pattern cleanly separates agent reasoning from tool execution, enabling independent testability of both layers.

The approval gate model ensures the platform augments human judgment rather than supplanting it: agents propose, operators decide, and the audit trail in `action_store` captures every outcome. Session-based credential management and resource group allowlisting provide a governance layer appropriate for multi-team Azure environments.

With the core read, analyze, and write-back capabilities fully operational, the platform is well-positioned for the next phase of development: persistent action history, multi-subscription support, and deeper FinOps intelligence. The architecture's modularity — separate route, service, agent, and tool layers — means each enhancement can be developed and tested in isolation without disrupting existing functionality.